

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN	25UAM083	Name	Adarsh Ingale
Division	B	Date	_____

Date and Time AI Agent

1. System Title & Short Description

A utility AI agent that interprets date and time requests and returns the required temporal information or calculation. It can handle current date/time, date arithmetic, date differences, and day-of-week queries.

2. Agent Purpose and Main Features

- Current date and time
- Date addition/subtraction
- Date difference
- Day-of-week calculation
- Readable formatted output

3. Context Diagram (Level 1)

The user supplies a temporal request. The agent parses the request, selects the appropriate operation, performs the calculation or retrieves the current local date/time, and formats the result for the user. The system clock is the only external system it depends on.

Level 1 - Context: Date and Time AI Agent



Figure 1: The user, the agent and the system clock.

4. Container Diagram (Level 2)

Level 2 - Containers: Date and Time AI Agent

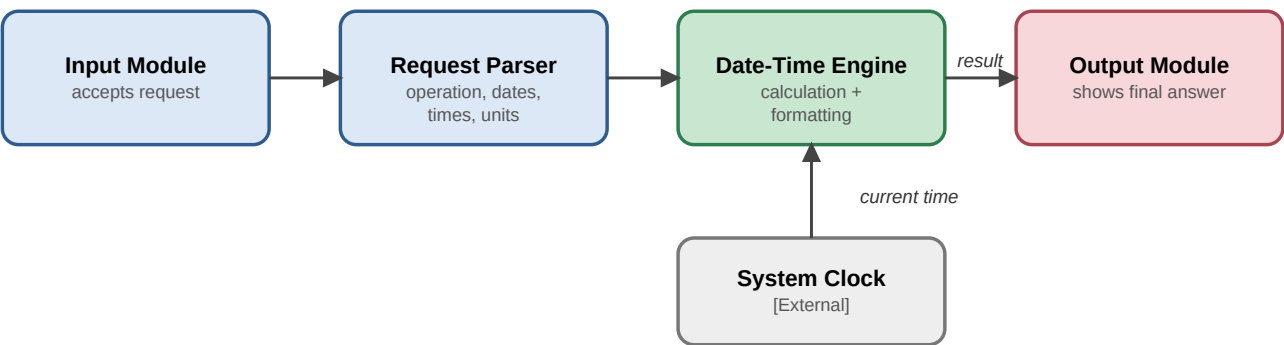


Figure 2: Main containers of the agent.

- **Input module:** accepts date/time requests.
- **Request parser:** identifies operation, dates, times, and units.
- **Date-Time engine:** performs calculations and formatting.
- **Output module:** presents the final answer.

5. Component Diagram (Level 3)

Level 3 - Components: inside the Date and Time AI Agent

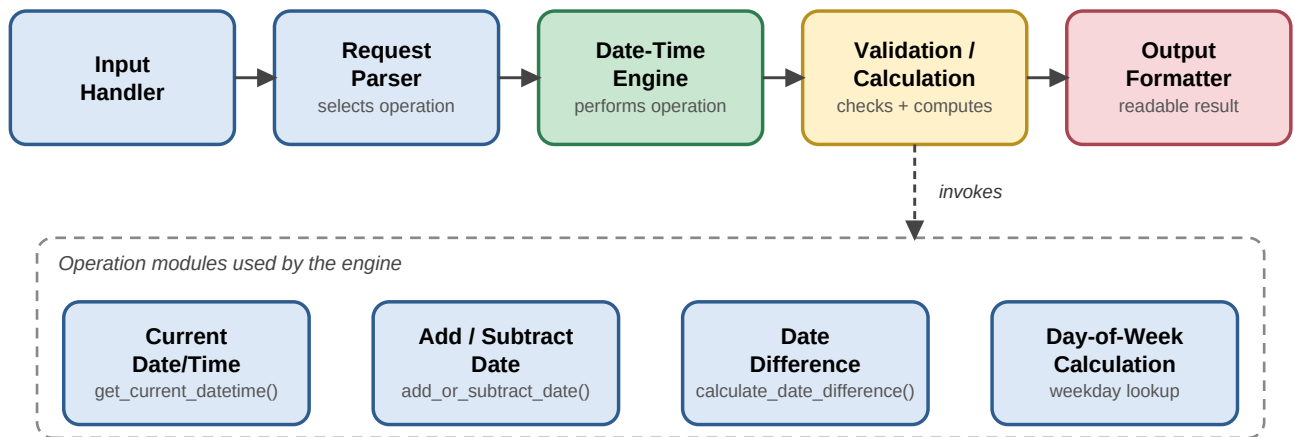


Figure 3: Components and the operation modules they use.

The major components are Input Handler, Request Parser, Date-Time Engine, Validation/Calculation component, and Output Formatter. The parser decides which operation is required; the engine performs it; the formatter produces the final response.

6. Code Level Overview (Level 4)

- `get_current_datetime()` – obtains current date and time.
- `parse_request(request)` – identifies the requested operation.
- `calculate_date_difference(date1, date2)` – calculates date difference.
- `add_or_subtract_date(date, value, unit)` – performs date arithmetic.
- `format_result(result)` – prepares readable output.

7. Design Decisions

The design keeps parsing, calculation, validation, and presentation separate. Standard date/time library functions should be used for reliable calendar calculations.

8. AI Contribution Note

AI tools can assist with code generation/refinement, test cases, documentation, and C4 diagrams. The student should select the functions, verify outputs, test edge cases, and explain the design.

9. Conclusion

The agent converts a user's temporal request into a structured operation and produces a useful result. The C4 architecture separates interaction, processing containers, components, and implementation functions.

Advanced Calculator AI Agent

1. System Title & Short Description

An advanced computational AI agent that accepts mathematical expressions and performs arithmetic and scientific calculations. It can support operations such as addition, subtraction, multiplication, division, powers, roots, percentages, trigonometric functions, and logarithms depending on the implemented functions.

2. Agent Purpose and Main Features

- Basic arithmetic
- Scientific functions
- Expression parsing
- Input validation
- Error handling
- Formatted results

3. Context Diagram (Level 1)

The user enters a mathematical expression or calculation request. The agent validates the input, parses the expression, selects the required mathematical operation, evaluates it, handles errors, and returns the result.

Level 1 - Context: Advanced Calculator AI Agent

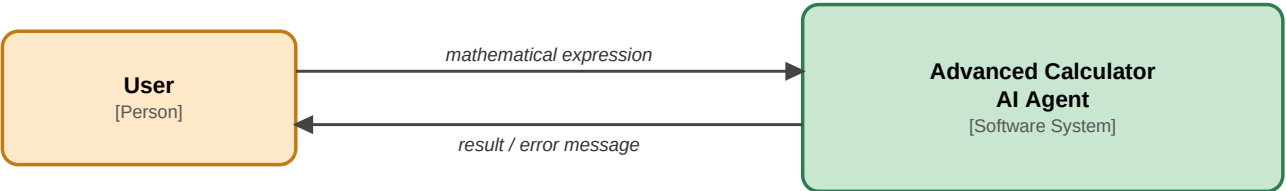


Figure 4: The user and the calculator agent.

4. Container Diagram (Level 2)

Level 2 - Containers: Advanced Calculator AI Agent

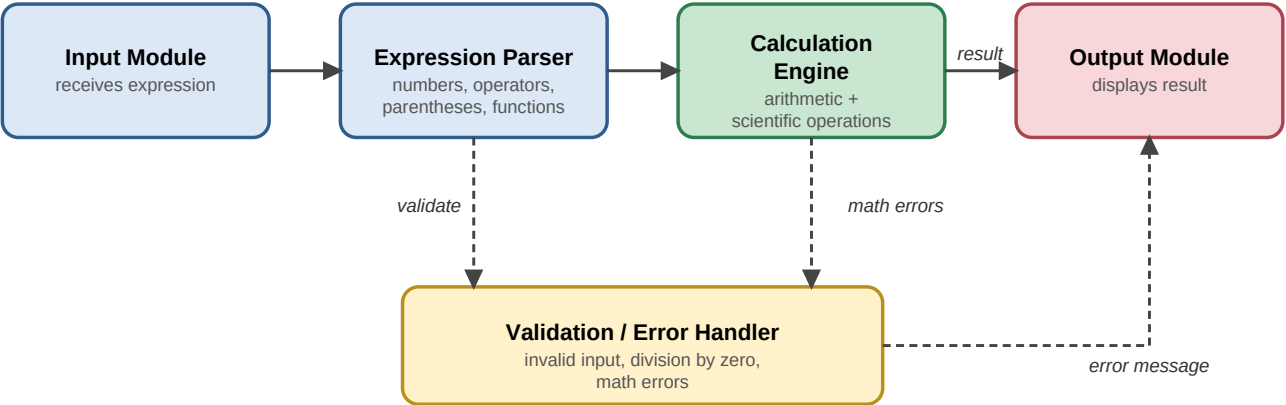


Figure 5: Main containers of the agent.

- **Input module:** receives the mathematical expression.
- **Expression parser:** identifies numbers, operators, parentheses, and functions.
- **Calculation engine:** evaluates arithmetic and scientific operations.
- **Validation/Error handler:** handles invalid input and mathematical errors.
- **Output module:** displays the result.

5. Component Diagram (Level 3)

Level 3 - Components: inside the Advanced Calculator AI Agent

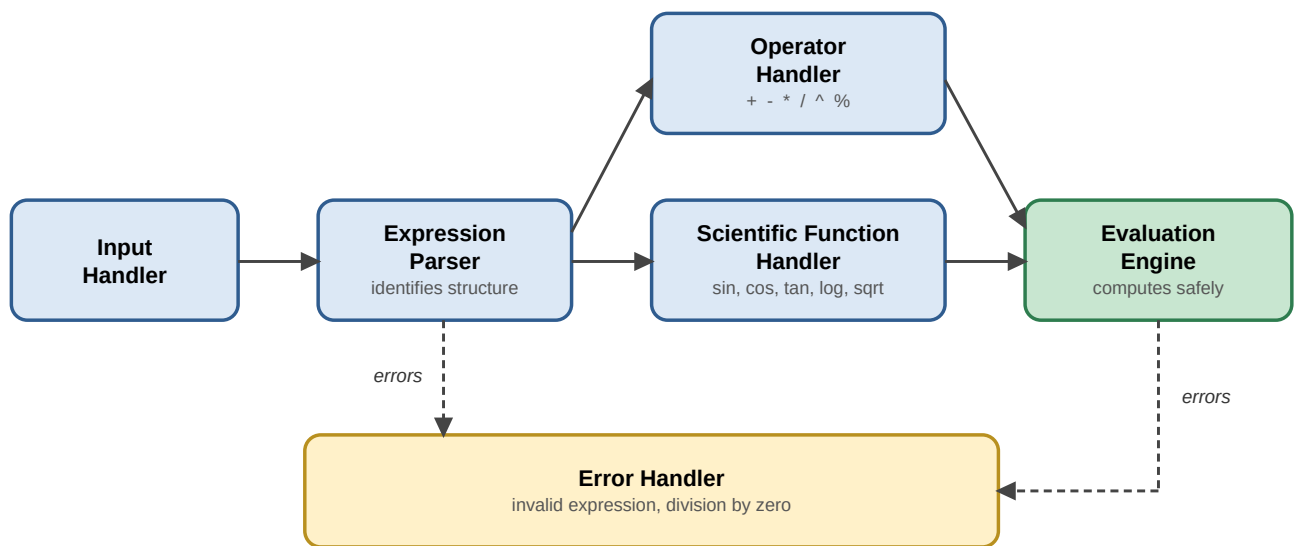


Figure 6: Components of parsing, evaluation and error handling.

The main components are Input Handler, Expression Parser, Operator Handler, Scientific Function Handler, Evaluation Engine, and Error Handler. Parsing identifies the structure of the expression, while the calculation engine evaluates it safely.

6. Code Level Overview (Level 4)

- **read_expression()** – accepts the user's calculation.
- **parse_expression(expression)** – identifies operands, operators, and functions.
- **evaluate_expression(expression)** – calculates the expression.
- **scientific_operation(value, operation)** – performs advanced functions.
- **validate_expression(expression)** – checks input errors.
- **display_result(result)** – presents the final answer.

7. Design Decisions

Expression parsing and evaluation are separated to make the system easier to test and extend. Validation is performed before evaluation so errors such as invalid expressions or division by zero can be reported clearly.

8. AI Contribution Note

AI tools can assist with code generation/refinement, test-case creation, documentation, and architectural diagrams. The student should decide supported operations, review the code, test expressions, and verify results.

9. Conclusion

The Advanced Calculator Agent demonstrates a modular computational workflow in which a user request becomes a mathematical operation and is evaluated by specialized components. The C4 model keeps the architecture consistent from system context to code-level functions.